

Contents

Topoliner. Short manual	1
1. Installation	1
1a. Interface language	1
2. Tools	1
3. Quick start	2
A polygon coverage	2
A line layer	2
4. Two thresholds	3
5. Tool 1.01 Polygon topology check	3
6. Tool 1.02 Line topology check	4
7. Tool 1.03 Polygon topology cleanup	4
8. Tool 1.04 Line topology cleanup	5
9. Tool 1.05 Node and vertex snapping	5
10. Tool 1.06 Insertion of missing nodes	6
11. Tool 1.07 Assembly check by attribute	6
12. Tool 2.01 Topology-preserving simplify	7
13. What is handled on shared borders	7
14. Z values	7
15. Warnings in the report	8
16. Limitations	8
17. Verifying the code	8

Topoliner. Short manual

[Русская версия](#)

Version 0.6.0

The plugin adds a **Topoliner** group with six tools to the Processing panel. Its purpose is to bring a polygon coverage into order without the manual routine of snapping a layer to itself.

1. Installation

Plugins - Manage and Install Plugins - Install from ZIP, choose topoliner.zip. Restarting QGIS is not required.

The tools appear in the **Processing** panel, group **Topoliner**.

1a. Interface language

The plugin is bilingual. The language is taken from the QGIS locale, not from the system one: if QGIS is set to English, the plugin interface is English too. Names, parameters, help texts, reports and violation names are all translated.

2. Tools

The tool number sets both the group and the order in the Processing panel: group **1. Topology** holds 1.01 to 1.06, group **2. Generalisation** holds 2.01. The order within the first group follows the workflow: check, clean, snap separately if needed, verify the assembly.

Tool	What it does	Modifies the layer
1.01 Polygon topology check	Finds violations, produces a point layer and a summary	No
1.02 Line topology check	Undershoots, overshoots, dangles, pseudo nodes	No
1.03 Polygon topology cleanup	Fixes everything that is debris	Yes, into a new layer
1.04 Line topology cleanup	Trims overshoots, closes undershoots, inserts nodes	Yes, into a new layer
1.05 Node and vertex snapping	Only reconciles nodes and vertices	Yes, into a new layer
1.06 Insertion of missing nodes	Adds nodes without changing shape or area	Yes, into a new layer
1.07 Assembly check by attribute	Checks whether groups assemble into one body. Polygons and lines	No
2.01 Topology-preserving simplify	Thins vertices of polygons and lines without tearing shared borders	Yes, into a new layer
2.02 Polygon borders as lines	Outputs borders as separate lines, each one once	No

All tools work in models and in batch mode. The input layer is never modified; the result always goes to a new layer.

3. Quick start

A polygon coverage

Four steps, in this order.

Step 1. Check. Run **1.01 Polygon topology check** with a tolerance of 2 and an area threshold of 1. Read the summary in the Processing panel. It answers the main question: how much will be fixed silently and how much has to be looked at by hand.

Step 2. Clean. Run **1.03 Polygon topology cleanup** with the same thresholds. Read the report, especially the line about the area.

Step 3. Check again. Repeat step 1 over the result. The **auto** column should be empty. Whatever remains under **review** is sorted out by hand.

Step 4. Check the assembly. Run **1.07 Assembly check by attribute** on the block field, then on the panel field. This is the acceptance criterion.

A line layer

Step 1. 1.02 Line topology check with a tolerance of 2. See how many undershoots and overshoots there are against dangles: the first two are fixed, the third is left to you.

Step 2. 1.04 Line topology cleanup with the same tolerance.

Step 3. Repeat step 1. Only dangles should remain.

Step 4. If needed, **1.07 Assembly check by attribute**: does a group of lines assemble into one connected chain.

4. Two thresholds

All behaviour is set by two numbers.

Tolerance, in layer CRS units. The maximum vertex displacement and the search radius for neighbouring edges. Take it slightly above the real size of the discrepancies and well below the width of the narrowest object. For borders digitised from plans this is usually 1 to 2 metres.

The main rule for the tolerance: it must be smaller than the width of the narrowest object. An object narrower than the tolerance has its opposite banks stick together and collapses into itself. Tool 1.05 prints the minimum and median ring width before the computation and reports how many rings are narrower than the tolerance. Such rings are left untouched by default and serve as anchors for their neighbours.

Area threshold, in square CRS units. The area below which a fragment is treated as technical debris. A reasonable starting point is the tolerance squared: with a tolerance of 2 metres that is 4 square metres, though 1 is often enough in practice.

Overlaps are judged by width, not by area. A strip as wide as the tolerance and tens of metres long collects a hundred square units while still being a consequence of vertex movement. Debris is an overlap narrower than the tolerance; a wider one is a dispute over real area.

If in doubt, start small: underestimating a threshold is safe, overestimating is not.

5. Tool 1.01 Polygon topology check

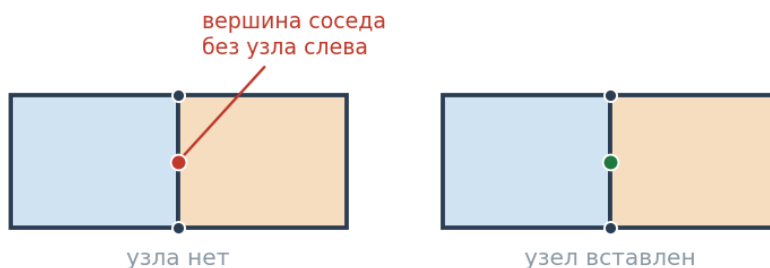


Figure 1: A neighbour vertex lies on the edge, but there is no node there.

Input. A polygon layer. Selected features only is supported.

Output. A point layer with the fields type, label, severity, fid_a, fid_b, value, note and grp.

How to read it. Style the layer by severity. The review points are the working list; the auto points do not need looking at, the cleanup tool will remove them.

Two similar violations are worth telling apart. *A vertex lies on a neighbour edge* means a distance of zero: the borders coincide geometrically but the neighbours have different vertices. That is a defect under any tolerance. *A vertex near an edge* means mere proximity, and the count is set entirely by the tolerance. If the median in the summary is close to the tolerance rather than to zero, you are looking at neighbouring objects, not at defects.

The grouping field is needed when the layer is not a single coverage, for example when zones of several seams share one layer. Such objects overlap by design, and without grouping every overlap lands in the findings. On a real zone layer of 379 objects a whole-layer check produced 166 findings, while grouping by seam gave 41.

The **cavity area threshold** separates a pillar, a lake or an unmapped area from a coverage defect: a small hole is an error, a very large one is almost always part of the design.

6. Tool 1.02 Line topology check



Figure 2: Three cases at a line end. Undershoots and overshoots are fixed, a dangle is left to the operator.

The layer is not modified. The output is a point layer with the findings and a summary in the Processing panel.

Lines have their own set of violations. Gaps, overlaps, nested objects and slivers do not apply to them, while undershoots, overshoots, dangles and pseudo nodes appear.

Three cases at a line end are worth telling apart. An *undershoot* is an end that did not reach a neighbouring line: the distance is below the tolerance, a digitising trace. An *overshoot* is a tail sticking out past an intersection, also a digitising trace. A *dangle* is an end with nothing to connect to: in a stream network or a set of mine workings it is usually an outlet or a dead end, so the decision is left to the operator.

The order of checks matters: the end of a tail past a node is also away from the neighbouring line, so overshoots are looked for before undershoots.

A **pseudo node** is a joint of two lines end to end with nothing else there. The check is off by default: in many layers the splitting into segments is deliberate.

7. Tool 1.03 Polygon topology cleanup

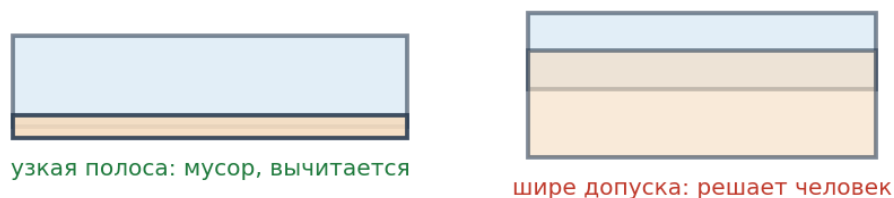


Figure 3: An overlap is judged by width, not by area.

Fixed silently: repeated vertices, spikes, vertices lying on a neighbour edge without a node, edge crossings without shared vertices, discrepancies below the tolerance, tiny holes and tiny parts, invalid geometry, narrow overlaps, small gaps.

Never done: wide overlaps, large gaps, duplicates, nested objects, sliver polygons, deletion of tiny objects without explicit permission. All of this goes to the **Remaining problems** layer with an explanation in note.

Order of steps. Vertex artefacts, then tiny parts and holes, then snapping, then repair of invalid geometry, then overlaps, then gaps. The order is fixed: spikes must go before snapping, otherwise they turn into false nodes and get spread across neighbours; validity repair must come after snapping, because snapping can itself produce a self-intersection.

Guarantees. No vertex moves further than the tolerance. An edit taking more than a quarter of an object's area is cancelled and the object is restored. A repeat run changes nothing. Attributes are preserved.

The output layer is always of multi type: a repair may split an object.

8. Tool 1.04 Line topology cleanup

Fixes what is certainly a digitising trace.

Removes repeated vertices and spikes, trims overshoots back to the intersection point itself, closes undershoots onto their projection on the neighbouring line, inserts missing nodes, removes zero-length lines.

What is not done: dangles are not touched, lines shorter than the threshold are not deleted without explicit permission, pseudo nodes are not merged.

Guarantee. A line end does not move further than the tolerance. A repeat run over the result changes nothing.

9. Tool 1.05 Node and vertex snapping

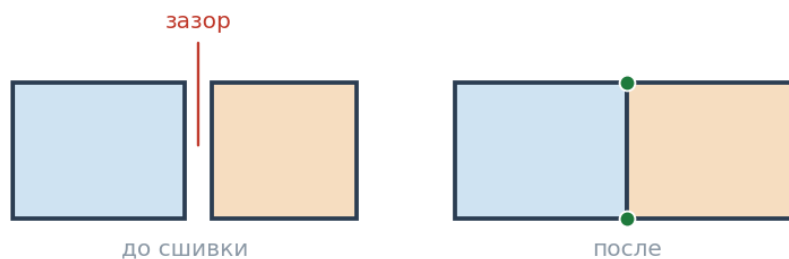


Figure 4: A gap below the tolerance is closed by merging vertices.

Use it when the coverage is otherwise fine and only the borders need reconciling. It does not touch overlaps or gaps.

A useful habit: run it first in **Insert nodes only** mode. It does not move a single existing coordinate and shows the real amount of discrepancy through the number of inserted nodes.

The reference layer is a separate parameter. Its vertices are treated as fixed and attract the input layer; the reference itself is not modified and does not appear in the result. Use it when a new area is stitched to a border that has already been accepted.

10. Tool 1.06 Insertion of missing nodes

Adds nodes where a vertex of one object lies on the edge of another. Nothing else.

How it differs from snapping. Snapping moves vertices to close discrepancies. Here there is no discrepancy at all: the borders already coincide, only the vertices differ. Insertion therefore moves nothing, and the total area stays exactly the same.

When it is needed. Different vertices do not affect rendering and are invisible to the eye, but they break dissolve by attribute, overlays, generalisation and loading into a database with strict topology. A typical source is independent construction of polygons on either side of a shared straight line, such as a fault line or the edge of a clipping mask.

Several passes may be required: an inserted node sometimes lands on the edge of a third object. The tool repeats passes while nodes are still found.

The deviation from the edge is not a tolerance. The distance here is zero, and the number only acts as a safeguard. The default matches coordinate precision; setting metres is wrong.

11. Tool 1.07 Assembly check by attribute

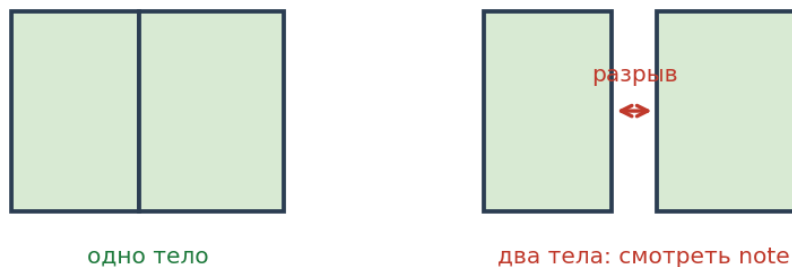


Figure 5: A group either assembles into one body or falls apart.

Checks that the union of objects sharing an attribute value gives exactly one part without interior rings. The layer is not modified.

What is found. *The group split into parts* means a break remains inside the group; the note field holds the distance to the nearest part, which is exactly the tolerance that was missing. *An interior ring within a group* means a hole remains inside.

An important property. The check finds what a coverage-wide gap search cannot: a gap reaching the outer edge is not a hole in the union, yet it cuts the group in two during assembly.

When a group need not be whole. One attribute value often describes several separated bodies: contour polygons of one level, islands, areas of one type. Set the **maximum gap** for such data. The sign that it is needed shows at once: if note holds hundreds of metres, these are not assembly defects.

12. Tool 2.01 Topology-preserving simplify

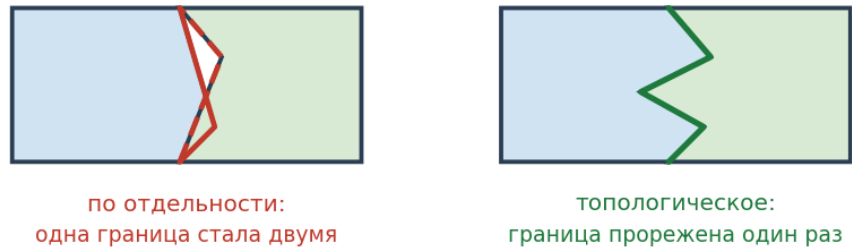


Figure 6: Independent simplification tears a shared border apart.

Thins vertices so that a border shared by two neighbours stays shared.

Why. Ordinary simplification processes each polygon separately, so the same border is thinned twice and differently, producing gaps and overlaps. Measured on a zone layer with a tolerance of 5: the topological version produces no violations at all, the independent one produces 46 overlaps, 25 gaps and 189 mismatched nodes.

Smoothing cuts corners after thinning, over the same arcs, so a shared border stays shared. Chaikin's scheme does not leave the original line, so no overshoots and no new self-intersections appear. Each pass roughly doubles the vertex count.

How it works. Shared edges are glued into arcs between branch nodes, each arc is thinned exactly once, and both neighbours receive the same result. Arc ends are fixed, so the nodes where three polygons meet do not move.

The area changes during simplification, that is its purpose. Topology is not damaged: check the result with 1.01, the number of findings must not grow.

13. What is handled on shared borders

Case	Result
A neighbour has a vertex on the shared border, we do not	A node is inserted, coordinates do not move
A T-junction of three polygons	The node appears where it was missing
Four polygons meeting at a corner within the tolerance	All corners merge into one point
A corner overshooting into a neighbour by less than the tolerance	Vertices merge, the overshoot disappears
Edges crossing without shared vertices	Nodes are placed at the intersections
A slanted border that already matches	Nothing changes, no false nodes
A mismatch larger than the tolerance	Left alone, this is a meaningful case

14. Z values

When snapping, a vertex takes the plan coordinates of the leader and keeps its own elevation, so seams with different Z do not get mixed.

In the cleanup tool the intersection operations work in plan, so Z is restored from the nearest original vertex. For coverages this is correct; for layers with sharp elevation changes along a border the result should be checked.

15. Warnings in the report

The total area changed by more than one per cent. Almost always means the area threshold is too high.

Degenerate rings removed. The tolerance exceeds the size of the objects themselves.

Objects disappeared. An object was lost entirely during repair. Find it in the remaining problems layer by the lost type.

Repairs cancelled due to area loss. The safeguard worked: repairing invalid geometry would have taken more than a quarter of the area, so the object was left unchanged.

Objects with ring self-touches. GEOS considers such geometry valid while SQL Server may reject it. Apply MakeValid on the server side when loading.

16. Limitations

Gaps are found as holes in the union of the coverage. A gap reaching the outer edge is not a hole and is not found this way. Such gaps are closed by vertex snapping when their width is below the tolerance.

The layer is read into memory in full. For layers of hundreds of thousands of objects, process them in parts.

Lines have their own group 3: check and cleanup. Lines are also accepted by 1.05 snapping, 1.06 node insertion, 2.01 simplification and 1.07 assembly check.

Automation cannot know which of two disputed borders is correct. It only tells scale: a centimetre overlap is an error, a hectare one is a disagreement between sources. Wide overlaps and large gaps therefore always stay with the operator.

17. Verifying the code

The geometric logic is kept out of QGIS and covered by tests:

```
cd topline  
python -m unittest discover -s tests -v
```

QGIS is not required for the tests. Overlap and gap checks are tested against Shapely, which uses the same GEOS as QGIS.